

DevPlan - Automated ROS Infrastructure Orchestration via LLM-Driven Templating - To refine

dan_marius.novischi@upb.ro

October 2025

1 Introduction and Motivation

Modern robotic systems such as the AntRobot operate within complex software ecosystems built on the Robot Operating System (ROS). These systems are modular and rely on interconnected nodes that handle perception, localization, mapping, planning, and control. Despite this modularity, configuring these nodes correctly remains a highly manual and error-prone process. Small changes—such as switching sensors, modifying mission modes, or adjusting parameters—can require extensive manual reconfiguration across YAML and launch files.

Such repetitive engineering tasks limit reproducibility, scalability, and experimentation speed in both research and applied robotics. Current orchestration solutions (e.g., ROScribe [1], RobotKube [2]) provide partial automation but still depend on static configuration schemes.

This research aims to address that gap by developing a hybrid orchestration framework that combines:

- **Templated Configuration Abstraction:** Using Jinja2 to convert static ROS configuration files into dynamic, reusable templates.
- **Language-Driven Orchestration:** Leveraging a Large Language Model (LLM) capable of translating high-level mission descriptions into structured, parameterized configurations.
- **Synthetic Data Bootstrapping:** Generating realistic mission–configuration pairs from a small set of real use cases to fine-tune the model.

The ultimate vision is a system that can understand a command such as:

“Deploy mapping with LiDAR and depth camera in exploration mode.”

and autonomously generate the necessary ROS configurations, parameter sets, and launch commands to execute the mission on the AntRobot.

2 State of the Art

2.1 ROS Orchestration and Configuration Management

Existing ROS orchestration frameworks aim to simplify deployment, but they focus mainly on containerization or node scheduling rather than dynamic configuration synthesis:

- **ROSScribe** [1] introduces a natural-language interface for generating launch commands but does not generate parameter files dynamically.
- **RobotKube** [2] leverages Kubernetes for distributed deployment but requires pre-defined configuration files.
- **OSRF Nexus** [3] offers dependency management but lacks semantic orchestration capabilities.

These tools improve reproducibility and scalability but remain static. They lack a reasoning layer capable of mapping human goals to specific ROS node graphs.

2.2 Templating Systems and YAML Automation

Templating engines such as Jinja2 [4] are widely used in infrastructure-as-code systems (Ansible, Terraform) for automating configuration file generation. They support conditional logic, variable substitution, and modular inclusion, making them ideal for defining flexible YAML configurations:

```
slam:
  use_lidar: {{ use_lidar | default(true) }}
  resolution: {{ resolution | default(0.05) }}
  frame_id: "{{ frame_id | default('map') }}"
```

By integrating Jinja2 with ROS, parameter files can be generated dynamically from a single, reusable structure—enabling mission-driven configuration generation.

2.3 Large Language Models for Structured Reasoning

LLMs such as Mistral [5] and DeepSeek [6, 7] demonstrate strong capabilities for code generation, structured reasoning, and schema adherence. Recent studies show that instruction-tuned models can consistently output valid JSON or YAML structures [8, 9]. This capability can be harnessed to map natural-language mission statements to structured configuration contexts that populate Jinja2 templates.

3 Proposed System Architecture

The proposed architecture (Figure 1) integrates three core layers:

1. **Templating Layer:** Encapsulates YAML and launch configurations in Jinja2 templates. Each node configuration becomes a reusable, parameterized component. Templates are stored hierarchically by node type (SLAM, localization, navigation).
2. **Language Understanding Layer:** Implements a fine-tuned LLM that interprets mission descriptions and outputs structured JSON contexts describing the required nodes, sensors, and parameters.
3. **Orchestration Layer:** Integrates the LLM and Jinja2 engine, validates configurations, and launches ROS stacks via Python launch APIs.

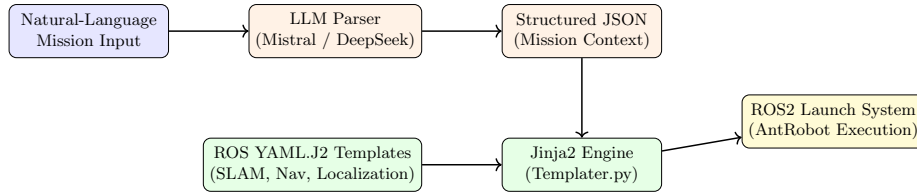


Figure 1: Proposed architecture of the LLM-driven orchestration system. The LLM interprets mission input, produces structured configuration contexts, and dynamically renders ROS parameter templates for automatic execution on the AntRobot.

4 Research Objectives

1. Conduct a detailed review of existing orchestration and templating approaches for ROS.
2. Develop a Jinja2-based templating system to abstract ROS configurations.
3. Design a data synthesis pipeline to generate mission-configuration examples.
4. Fine-tune and evaluate LLMs (Mistral, DeepSeek) for mission-to-configuration translation.
5. Integrate and validate the system on both simulated and physical AntRobot platforms.

5 Methodology and Development Plan

The project will be implemented over four semesters, following a structured sequence of theory, development, experimentation, and validation.

Semester 1: Foundation — State of the Art, Architecture, and Jinja2 Templating Demonstration

Aim: Build the conceptual and technical foundation through a detailed state-of-the-art study, system design, and the creation of a reusable templating engine demonstrated on the AntRobot.

Tasks:

1. Conduct literature review on orchestration frameworks, YAML automation, and LLM code synthesis.
2. Produce comparative analysis tables summarizing limitations and advantages of existing systems.
3. Design system architecture diagram, defining data interfaces between modules.
4. Convert AntRobot’s YAML files (SLAM, localization, navigation, perception) into Jinja2 templates.
5. Implement a Python module (`templater.py`) capable of rendering parameterized configurations:

```
def render_config(template_name, context):
    env = Environment(loader=FileSystemLoader('templates'))
    tpl = env.get_template(template_name)
    return yaml.safe_load(tpl.render(context))
```

6. Create a usage guide (README) detailing folder structure, template syntax, and variable naming conventions.
7. Validate generated YAMLs via AntRobot simulation and hardware tests:
 - Demo 1 — Template-based SLAM configuration launch.
 - Demo 2 — Mission change: Mapping → Mapping & Navigation.
 - Demo 3 — Dynamic parameter change (LiDAR → Stereo camera (maybe)).

Deliverables:

- Literature review report and comparative matrix.
- System architecture document and diagrams.
- Functional templating engine with documentation.
- AntRobot demos showing successful dynamic configuration launches.

Semester 2: Synthetic Data Generation and Initial LLM Integration

Aim: Develop a synthetic bootstrapping pipeline to generate structured mission–configuration pairs and connect the LLM to the templating system.

Tasks:

1. Define 10–15 representative missions from AntRobot’s operational modes.
2. Generate paraphrased mission variants using Mistral or DeepSeek to expand data to 100+ examples.
3. Convert mission–YAML mappings into structured JSONL format:

```
{"instruction": "Run SLAM with LiDAR.",  
  "output": {"nodes": ["slam", "lidar_driver"],  
             "parameters": {"use_lidar": true}}}
```

4. Integrate Hugging Face Transformers for LLM inference:

```
from transformers import pipeline  
gen = pipeline("text-generation", model="mistralai/Mistral-7B-Instruct-v0.2")
```

5. Connect JSON outputs to the templating system to produce launchable YAMLs.
6. Validate generated configurations on the AntRobot.

Deliverables:

- Synthetic dataset (`missions.jsonl`) with mission–configuration pairs.
- LLM integration prototype for text-to-YAML generation.
- Working demonstration of LLM-assisted configuration rendering.

Semester 3: Model Fine-Tuning, RAG Integration, and Benchmarking

Aim: Improve the LLM's precision and reliability using fine-tuning and retrieval-augmented generation.

Tasks:

1. Expand the dataset with new missions and real-world examples collected during testing.
2. Fine-tune Mistral or DeepSeek using PEFT and QLoRA:

```
from peft import LoraConfig, get_peft_model
config = LoraConfig(r=8, lora_alpha=32, target_modules=["q_proj"])
```

3. Develop a retrieval-augmented generation module (LangChain + ChromaDB) indexing ROS documentation.
4. Evaluate zero-shot, few-shot, fine-tuned, and RAG-enhanced performance across:
 - JSON validity rate
 - Template fill accuracy
 - Launch success percentage

Deliverables:

- Fine-tuned model adapter.
- RAG-enabled orchestration module.
- Comprehensive evaluation and ablation study report.

Semester 4: System Integration, Validation, and Thesis Completion

Aim: Integrate all modules into a single operational pipeline, validate on real hardware, and finalize documentation and thesis.

Tasks:

1. Integrate LLM interpreter, templating engine, and ROS orchestrator into a unified interface:

```
$ python orchestrate.py "Run mapping with LiDAR and camera"
```

2. Conduct experiments on Gazebo simulation and AntRobot hardware.
3. Quantitatively evaluate:
 - Time saved per configuration vs manual setup.
 - YAML schema and parameter validity rate.
 - Mission completion success rates.
4. Prepare full thesis, results, and publication for submission.

Deliverables:

- Fully integrated orchestration framework.
- Validation results and performance benchmarks.
- Master’s thesis and oral defense.

6 Expected Contributions

- A fully documented, reusable orchestration framework linking natural-language mission intent to ROS configuration execution.
- A synthetic dataset and methodology for small-sample fine-tuning of LLMs in robotics.
- A comparative study of LLM reasoning strategies (zero-shot, fine-tuning, RAG) applied to configuration synthesis.
- A measurable improvement in deployment efficiency and robustness for the AntRobot.

References

- [1] R. Technologies, “Roscribe: Natural language interface for ros task configuration,” <https://github.com/RoboCoachTechnologies/ROSScribe>, 2025.
- [2] R. A. University, “Robotkube: Kubernetes-based robotic infrastructure management,” <https://github.com/ika-rwth-aachen/robotkube>, 2025.
- [3] O. S. R. Foundation, “Osrif nexus: Ros dependency and version management,” <https://github.com/osrf/nexus>, 2025.
- [4] P. Projects, “Jinja2 documentation,” <https://jinja.palletsprojects.com>, 2025.

- [5] A. Mensch, T. Le Scao, G. Lample, and et al., “Mistral 7b: A 7-billion-parameter language model engineered for superior performance and efficiency,” *arXiv preprint arXiv:2310.06825*, 2023. [Online]. Available: <https://arxiv.org/abs/2310.06825>
- [6] D. Guo, Q. Zhu, D. Yang, and et al., “Deepseek-coder: When the large language model meets programming – the rise of code intelligence,” *arXiv preprint arXiv:2401.14196*, 2024. [Online]. Available: <https://arxiv.org/abs/2401.14196>
- [7] D. Guo, D. Yang, H. Zhang, and et al., “Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning,” *arXiv preprint arXiv:2501.12948*, 2025. [Online]. Available: <https://arXiv.org/abs/2501.12948>
- [8] B. Agarwal, I. Joshi, and V. Rojkova, “Think inside the json: Reinforcement strategy for strict llm schema adherence,” *arXiv preprint arXiv:2502.14905*, 2025. [Online]. Available: <https://arXiv.org/abs/2502.14905>
- [9] J. Xie, S. Syu, and H.-y. Lee, “Exploring the impact of instruction-tuning on llm’s susceptibility to hallucinations,” in *Proceedings of ACL Long*, 2025, p. –. [Online]. Available: <https://aclanthology.org/2025.acl-long.1295.pdf>